

Tugas Praktikum OOP - Modul 5

Inheritance, Generalization, Superclass, dan Subclass

Nama : Rafael Rizky
NRP : 3125522007

Dosen : Nirwana Haidar Hari, S.Pd., M.Kom.
Kampus : PENS PSDKU Sumenep (2026)

HALAMAN 1 — ANALISIS, SPRINT PLANNING & AUDIT GENERALIZATION

1. **Nama Project:** Sistem Kasir Sederhana
2. **Sprint Goal:** Memperbaiki desain proyek dengan mengidentifikasi class yang memiliki karakteristik serupa, melakukan generalization untuk membentuk Superclass Produk, serta mengimplementasikan Subclass ProdukMakanan dan ProdukElektronik menggunakan klausa extends dan super, dengan tetap mempertahankan seluruh relasi P4.

3. Sprint Backlog (P5)

ID	Sprint Backlog	Status
SB-01	Mencari dan mengaudit class yang memiliki duplikasi atribut/behavior	DONE
SB-02	Menentukan dan mendefinisikan Superclass (Produk)	DONE
SB-03	Menentukan dan mendefinisikan Subclass (ProdukMakanan dan ProdukElektronik)	DONE
SB-04	Memperbarui UML Class Diagram proyek sebelum dan sesudah refactoring	DONE
SB-05	Implementasi pewarisan menggunakan kata kunci extends pada subclass	DONE
SB-06	Implementasi pemanggilan constructor superclass menggunakan super(...)	DONE
SB-07	Melakukan pengujian inheritance pada Main.java (transaksi multi-kategori)	DONE

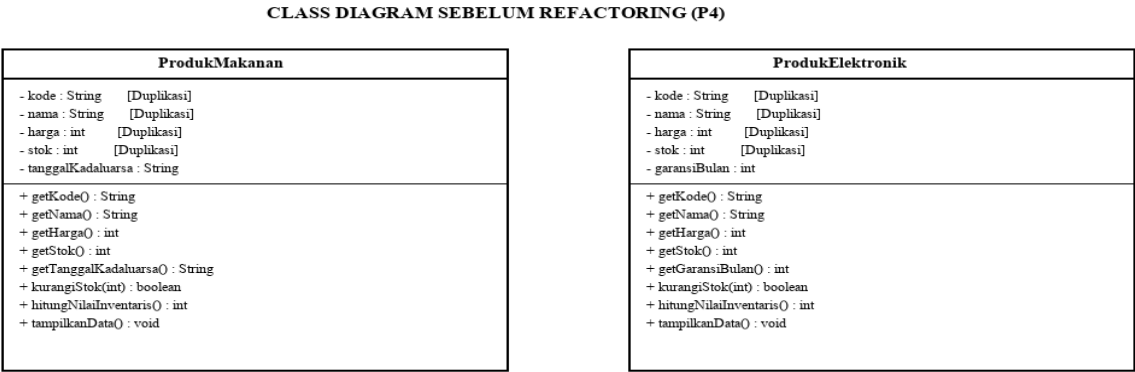
4. Audit Class P4 & Kandidat Generalization (Bagian A)

Class	Attribute Umum (Kandidat Superclass)	Attribute Khusus (Subclass)
ProdukMakanan	kode, nama, harga, stok	tanggalKadaluarsa (String)
ProdukElektronik	kode, nama, harga, stok	garansiBulan (int)

5. Kandidat Superclass dan Subclass:

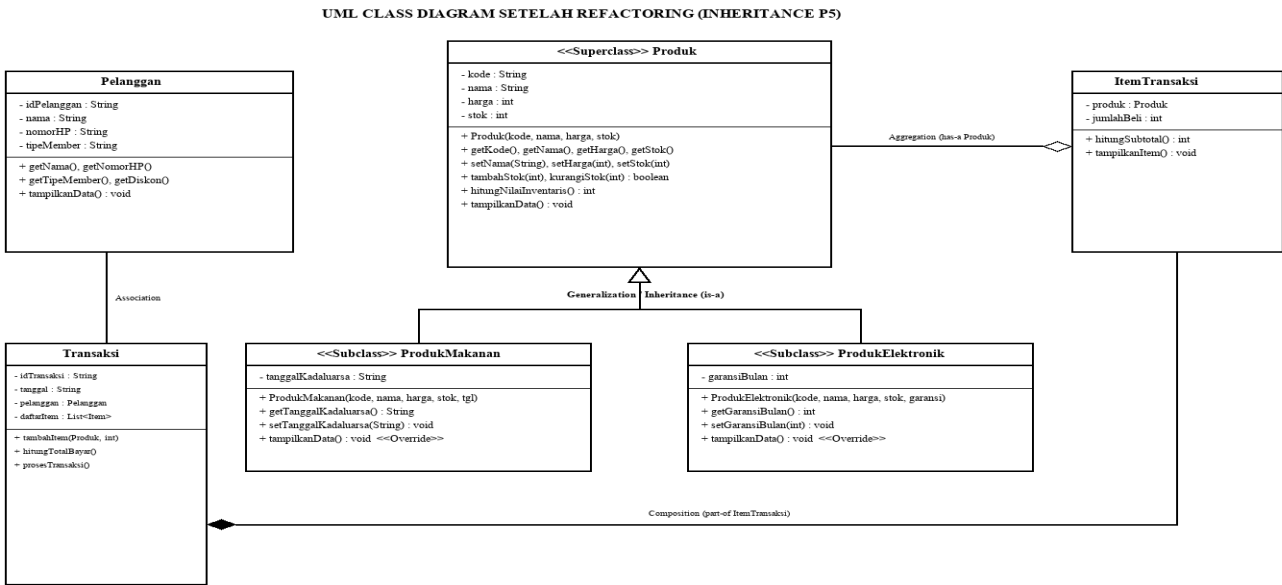
- **Superclass: Produk** (menampung atribut bersama: kode, nama, harga, stok, serta method kurangiStok, tambahStok, hitungNilaiInventaris).
- **Subclass 1: ProdukMakanan** (mewarisi Produk, menambahkan atribut tanggalKadaluarsa).
- **Subclass 2: ProdukElektronik** (mewarisi Produk, menambahkan atribut garansiBulan).

6. Class Diagram Sebelum Refactoring (P4)



Gambar 1. UML Class Diagram Sebelum Refactoring (Terjadi Duplikasi Atribut)

7. Class Diagram Setelah Refactoring (P5: Dengan Inheritance)



Gambar 2. UML Class Diagram Setelah Refactoring (Penerapan Inheritance & Generalization P5)

8. Penjelasan Hubungan is-a dan Manfaat Refactoring:

- **Hubungan is-a:** *ProdukMakanan* is-a *Produk* dan *ProdukElektronik* is-a *Produk*. Keduanya merupakan jenis produk yang valid dengan atribut kode, nama, harga, dan stok yang sama.
- **Manfaat Refactoring:** (1) Menghilangkan duplikasi kode, (2) Mempermudah perawatan sistem secara modular, (3) Memungkinkan *ItemTransaksi* dan *Transaksi* dari P4 menampung seluruh turunan *Produk* secara polimorfik tanpa perlu perombakan kode.

9. Screenshot Hasil Running Program (Main.java)



Gambar 3. Bukti Running Program Main.java pada Terminal (Eksekusi Pengujian P5)

10. Bukti Penggunaan extends dan super:

- **extends:** Dideklarasikan pada subclass: public class ProdukMakanan extends Produk dan public class ProdukElektronik extends Produk.
- **super(...):** Digunakan pada constructor subclass untuk meneruskan argumen dasar ke Superclass: super(kode, nama, harga, stok);. Urutan eksekusi: Constructor Produk dieksekusi terlebih dahulu untuk validasi awal, kemudian constructor subclass dieksekusi untuk inialisasi atribut khusus.

11. Sprint Review

Item	Hasil
Kandidat inheritance ditemukan	Ditemukan entitas ProdukMakanan dan ProdukElektronik dengan atribut bersama.
Superclass berhasil dibuat	Superclass Produk berhasil didefinisikan dengan atribut umum dan method mutator.
Minimal 2 subclass dibuat	Dibuat 2 subclass: ProdukMakanan dan ProdukElektronik.
extends diterapkan	Klausula extends diterapkan pada kedua subclass untuk mewarisi class Produk.
super diterapkan	super(...) diterapkan pada constructor subclass untuk inialisasi superclass.
Class diagram diperbarui	Class diagram diperbarui dengan simbol generalization dan perbandingan sebelum/sesudah.
Program berhasil dijalankan	Kompilasi (javac) dan eksekusi (java) berhasil 100% tanpa error.
Kendala	Tidak ada kendala fatal. Seluruh relasi P4 terintegrasi mulus dengan inheritance P5.

12. Sprint Retrospective

What Went Well? Seluruh konsep generalization dan inheritance berhasil diimplementasikan secara elegan, mengeliminasi duplikasi kode dan memperkuat modularitas kasir.

What Went Wrong? Diperlukan kehati-hatian dalam sinkronisasi constructor agar urutan pemanggilan super(...) memenuhi kaidah Java.

Improvement: Pada sprint berikutnya (P6: Polymorphism), akan dikembangkan dynamic method overriding untuk perhitungan diskon dan tampilan struk yang lebih variatif.